COMP2271: Computer Graphics

Part 2: Transformations

Lecturer: Frederick Li
Department of Computer Science
Durham University

Abstract

This handout covers the fundamental principles of 2D and 3D transformations in computer graphics. We begin by exploring the basic 2D operations of translation, rotation, and scaling. We then introduce the concept of representing these transformations using matrices and the need for homogeneous coordinates, which provide a unified mathematical framework. Building on this foundation, we extend these ideas into three dimensions, introducing the Model, View, and Projection matrices. The combination of these matrices forms the Model-View-Projection (MVP) matrix, a cornerstone of the modern graphics pipeline that transforms vertices from their local model space all the way to the 2D screen space. Throughout, we will connect these theoretical concepts to practical implementation using 'three.js', a popular WebGL library.

How to Learn This Lecture

Transformations are the grammar of computer graphics. To master them, focus on these three mental shifts:

- **Do Not Memorise the Grid:** You do not need to memorise the 4×4 matrix of numbers for a rotation. Instead, understand the *concept* of how multiplying a matrix by a vector modifies that vector.
- **Respect the Order (Right-to-Left):** This is the most common source of bugs. Always read transformation chains backwards. $M \cdot V \cdot P$ means "Transform the Point (P) by the Model Matrix (M), then the View (V)..." – Wait! In standard math notation, it is $P_{final} = M_{proj} \cdot M_{view} \cdot M_{model} \cdot P_{vertex}$. The vertex is on the far right, so it hits the Model matrix first.
- **Visualise the Spaces:** Do not think of coordinates as static numbers. Think of "Spaces." An object starts in its own private universe (Model Space). We move that universe into the world (World Space), then we move the world to align with the camera (View Space).

1 Section 1: 2D Transformations

1.1 Introduction to Transformations

In the previous part, we established the basics of the rendering pipeline, where we can take raw vertex data and produce a 2D image. However, the objects we could render were static; their vertices were defined with fixed positions. To create dynamic and interactive scenes, we need to be able to move, rotate, and resize these objects. These operations are collectively known as **transformations**.

A naive approach might be to modify the vertex data directly on the CPU. For example, to move an object, we could iterate through every one of its vertices and add a value to their x and y coordinates. This seems intuitive, but is incredibly inefficient for several reasons:

- **High CPU Cost:** For a model with thousands or millions of vertices, performing these calculations on the CPU every frame would be a significant performance bottleneck. The CPU has other important tasks to manage, such as game logic, physics, and AI.
- **Data Transfer:** The bus between the CPU and GPU is a limited resource. Re-uploading the entire, modified vertex buffer to the GPU every single frame is a slow operation that would cripple performance. Imagine having to send a brand new blueprint of a car to the factory just to change the position of the side mirror.
- **Loss of Original Data:** The original model data would be overwritten. This makes it difficult to revert to the original state or to perform calculations based on the object's local, unmodified shape.

A much more powerful and efficient method is to perform these transformations on the GPU within the **Vertex Shader**. We send the original, unmodified vertex data to the GPU once, and then for each frame, we send a small, compact set of instructions that tells the GPU *how* to

transform the vertices. These instructions are encoded in a mathematical tool called a **transformation matrix**. This approach offloads the heavy parallel work to the hardware designed for it (the GPU) and minimises the data transfer between CPU and GPU.

1.2 Fundamental 2D Transformations

There are three fundamental transformations that form the basis of most 2D graphics operations.

1.2.1 Translation (Moving)

Translation is the process of moving an object from one position to another. It is defined by a **translation vector** $T = (t_x, t_y)$, which specifies the displacement in the x and y directions. For any vertex $P = (x, y)$, the translated vertex P' is calculated by simple vector addition:

$$P' = P + T = (x + t_x, y + t_y)$$

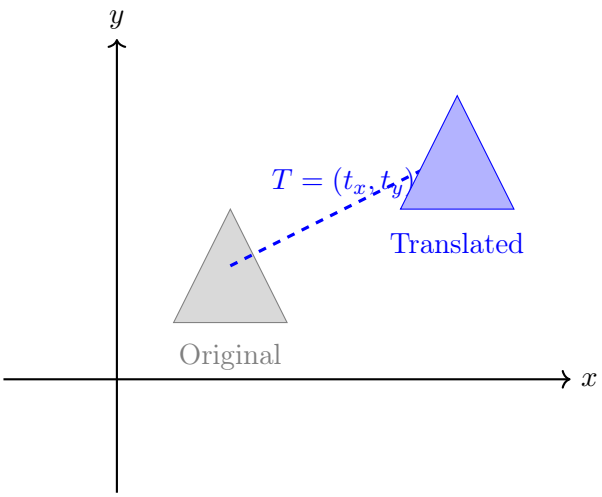

Figure 1: A 2D translation applied to a triangle. Every vertex is shifted by the same vector T.

1.2.2 Rotation (Turning)

Rotation is the process of turning an object around a specific point, known as the **pivot point**. For simplicity, we first consider rotation around the origin $(0, 0)$. To rotate a vertex $P = (x, y)$ by an angle θ anti-clockwise, we use the following trigonometric formulae to find the new vertex $P' = (x', y')$:

$$x' = x\cos(\theta) - y\sin(\theta)$$
$$y' = x\sin(\theta) + y\cos(\theta)$$

This is a fundamental result from trigonometry. The derivation involves representing the point (x, y) in polar coordinates, where its position is defined by a radius r and an angle ϕ. Rotating the point by θ simply means changing its angle to $\phi + \theta$, and these formulae are the result of converting back to Cartesian coordinates.

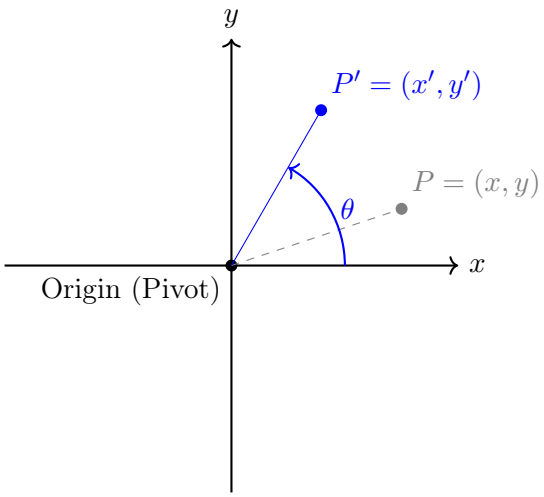

Figure 2: A 2D rotation of a point around the origin. The point maintains its distance from the pivot.

1.2.3 Scaling (Resizing)

Scaling is the process of changing the size of an object. It is defined by a **scaling vector** $S = (s_x, s_y)$. For a vertex $P = (x, y)$, the scaled vertex $P' = (x', y')$ is found by component-wise multiplication:

$$x' = x \cdot s_x$$
$$y' = y \cdot s_y$$

If $s_x = s_y$, the scaling is **uniform**, preserving the object's aspect ratio. If $s_x \neq s_y$, the scaling is **non-uniform**, which will stretch or squash the object. Scaling is also performed relative to the origin; vertices move farther away from or closer to the origin based on the scaling factors.

1.3 Representing Transformations with Matrices

While the equations above work, they are all different operations (addition for translation, trigonometric functions for rotation, multiplication for scaling). This is inconvenient. We need a single, consistent mathematical operation to represent all transformations. Linear algebra provides this unified way: the **matrix**.

The key idea is to represent a 2D point (x, y) as a column vector $\begin{pmatrix} x \\ y \end{pmatrix}$ and a transformation as a matrix. The transformation is then applied by performing matrix-vector multiplication.

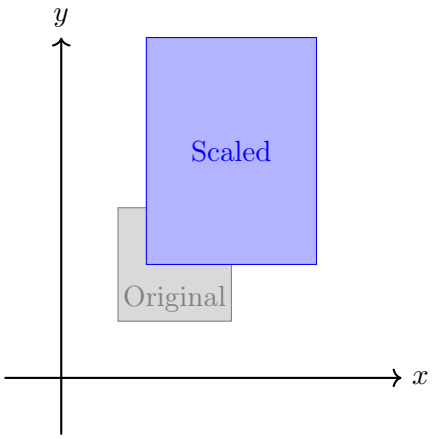

Figure 3: A non-uniform 2D scaling with $S = (1.5, 2.0)$. The object is stretched more vertically than horizontally.

Scaling Matrix: The scaling operation $x' = x \cdot s_x, y' = y \cdot s_y$ can be written in matrix form as:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Rotation Matrix: The rotation operation $x' = x\cos\theta - y\sin\theta, y' = x\sin\theta + y\cos\theta$ can be written as:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

This is a very powerful unification. Both scaling and rotation are now a single, consistent operation: multiplication by a 2x2 matrix. But what about translation?

1.4 The Problem with Translation

Translation is an addition: $P' = P + T$. There is no 2x2 matrix that can be multiplied by $\begin{pmatrix} x \\ y \end{pmatrix}$ to produce $\begin{pmatrix} x + t_x \\ y + t_y \end{pmatrix}$. This is because matrix multiplication on a 2D vector is a *linear* transformation, which can scale, rotate, or shear, but it must always leave the origin $(0, 0)$ unchanged. Since translation moves every point, including the origin, it is an *affine* transformation, not a linear one. This is a problem because we want a single, unified system. If we can't represent translation as a matrix multiplication, we can't easily combine it with rotation and scaling into a single matrix. This would defeat the purpose of our GPU-efficient pipeline.

1.5 Homogeneous Coordinates

To solve the translation problem, we use a clever mathematical trick known as **homogeneous coordinates**. The idea is to embed our 2D space into a 3D space. We represent a 2D point

$P = (x, y)$ as a 3D vector by adding a third 'w' coordinate and setting it to 1: $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$. This seems strange, but by moving to 3D vectors and 3x3 matrices, we can now use the third column of the matrix to encode translation. This effectively turns the linear operation of matrix multiplication in 3D into an affine operation in 2D.

Translation Matrix (in Homogeneous Coordinates):

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

If you perform this multiplication, you get:
$x' = 1 \cdot x + 0 \cdot y + t_x \cdot 1 = x + t_x$
$y' = 0 \cdot x + 1 \cdot y + t_y \cdot 1 = y + t_y$
$1 = 0 \cdot x + 0 \cdot y + 1 \cdot 1 = 1$
This successfully performs the translation.

We must also update our scaling and rotation matrices to be 3x3 so they can operate on these new 3D vectors. We do this by embedding the 2x2 matrix inside a 3x3 *identity* matrix.

Scaling Matrix (Homogeneous):

$$\begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation Matrix (Homogeneous):

$$\begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Now, all three fundamental 2D transformations can be represented by multiplication with a 3x3 matrix. This is a crucial breakthrough for creating an efficient and unified graphics pipeline.

1.6 Combining Transformations

The real power of using matrices is that we can combine a sequence of transformations into a single matrix. For example, if we want to first scale an object, then rotate it, and finally translate it, we would apply the matrices in that order:

$$P' = M_{translate} \cdot M_{rotate} \cdot M_{scale} \cdot P$$

The key insight is that we can pre-multiply the transformation matrices together on the CPU to form a single, combined matrix:

$$M_{combined} = M_{translate} \cdot M_{rotate} \cdot M_{scale}$$

This calculation is done only once per frame (or only when the object moves). Then, in the vertex shader, we only need to perform a single matrix multiplication for each vertex:

$$P' = M_{combined} \cdot P$$

This is incredibly efficient. We can send one combined matrix to the GPU, and it can apply a complex series of transformations to millions of vertices with just one multiplication each.

1.6.1 The Importance of Order

Matrix multiplication is **not commutative**. This means that $A \cdot B \neq B \cdot A$. The order in which you apply transformations matters significantly. In computer graphics, transformations are applied from right to left (as the vertex vector P is on the far right). Consider the difference between "rotate then translate" and "translate then rotate".

- **Rotate, then Translate ($M_{translate} \cdot M_{rotate} \cdot P$):** The object first rotates around the world origin $(0, 0)$ and then moves to its final position. This is usually the desired behaviour for placing an object in the world.
- **Translate, then Rotate ($M_{rotate} \cdot M_{translate} \cdot P$):** The object first moves away from the origin and then rotates around the world origin. This causes the object to "orbit" the origin, which is usually not what is intended.

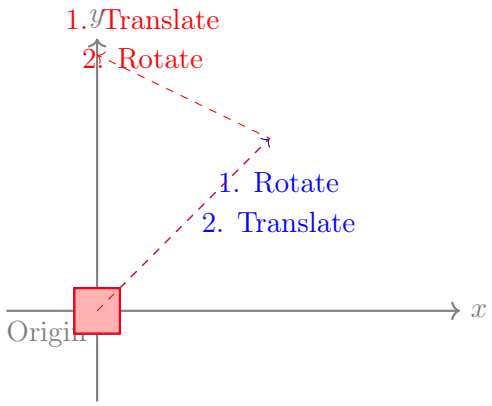

Figure 4: Visualising the importance of transformation order. The blue square was rotated at the origin, then translated. The red square was translated, then rotated around the origin, causing it to orbit into an unexpected position.

The standard order of operations for placing an object is **Scale, then Rotate, then Translate** ($T \cdot R \cdot S$). This ensures the object scales in its own local space, rotates around its own centre, and then moves to its final world position.

1.7 Time-Based Animation

Animation is achieved by changing transformation values over time within the application's **render loop**. A simple approach is to change a value by a fixed amount each frame.

```
let currentRotation = 0.0;

function drawScene() {
  // Increment by a fixed amount each frame
  currentRotation += 0.01;
  // ... update matrix and draw ...

  requestAnimationFrame(drawScene);
}
```

Listing 1: Frame-based animation (Incorrect)

The problem with this method is that the animation speed becomes dependent on the user's frame rate. A user with a 144Hz monitor will see the object rotate much faster than a user with a 60Hz monitor because 'drawScene' is called more often. This leads to an inconsistent and unfair user experience.

The correct solution is **time-based animation**. We calculate the time elapsed since the last frame (delta time) and update our animation values based on that. This ensures the animation speed is consistent across all hardware.

```
let then = 0;
let rotation = 0.0;
const rotationSpeed = 1.5; // radians per second

function drawScene(now) {
  now *= 0.001; // convert timestamp to seconds
  const deltaTime = now - then;
  then = now;

  // Update rotation based on elapsed time
  rotation += rotationSpeed * deltaTime;

  // ... update matrix and draw ...

  requestAnimationFrame(drawScene);
}
```

Listing 2: Time-based animation (Correct)

This approach decouples the animation speed from the rendering speed, resulting in a smooth and consistent experience for all users.

2 Section 2: 3D Transformations and The Camera

2.1 Extending to Three Dimensions

Moving from 2D to 3D is a natural extension of the concepts we have just learned. The principles remain the same, we just add an extra dimension.

- Our vertices now have three components: (x, y, z).
- We use homogeneous coordinates by adding a 'w' component, making our vectors 4D: $(x, y, z, 1)$.
- Consequently, our transformation matrices are now 4x4.

The process of taking a 3D model and rendering it onto our 2D screen is a journey through several distinct **coordinate systems**. This journey is orchestrated by a sequence of three critical matrices.

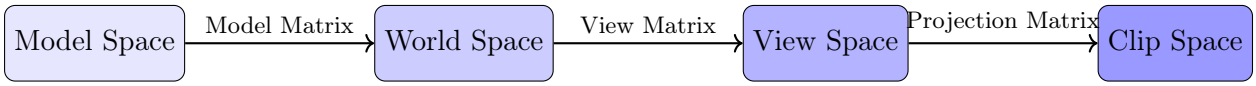

Figure 5: The transformation pipeline from 3D model space to 2D screen space.

2.2 The Model Matrix

A 3D model is typically created in its own local coordinate system, called **Model Space**. For example, an artist might model a character centred at the origin $(0, 0, 0)$ and standing upright along the y-axis. This is the most convenient way to work on the model in isolation. The **Model Matrix** is responsible for taking these model-space vertices and positioning them within the larger 3D world. It applies a specific translation, rotation, and scale to the object to set its size, orientation, and position in the overall scene. This transforms the vertices from Model Space into **World Space**. Every object in your scene will have its own unique Model Matrix.

The 3D transformation matrices (in homogeneous coordinates) are direct extensions of their 2D counterparts:

$$M_{translate} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad M_{scale} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation in 3D is more complex as we can rotate around the X, Y, or Z axes.

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R_y(\theta) = \begin{pmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

The final Model Matrix is a combination of these, typically in the order T · R · S.

2.3 The View Matrix (The Camera)

Now that all our objects are positioned in World Space, we need to view them from a specific point. This is the job of the **camera**.

A common misconception is that we move the camera around the world. While conceptually true, in practice it is mathematically easier to do the opposite: **we move and rotate the entire world to position it in front of a fixed camera**. Imagine holding a snow globe: you don't walk around the globe to see the other side; you simply turn the globe itself.

We transform the world coordinates into **View Space** (also called Camera Space), where:

- The Camera is at the origin $(0, 0, 0)$.
- The Camera is looking down the negative Z-axis $(-Z)$.

2.3.1 Mathematical Formulation: The Inverse Matrix

To construct this matrix, we define the camera's local coordinate system using three orthogonal unit vectors (its basis):

- **eye**: The absolute position of the camera in world space (eye_x, eye_y, eye_z).
- **f** (Forward): The direction vector the camera is looking (maps to $-Z$).
- **r** (Right): The direction vector pointing to the camera's right (maps to $+X$).
- **u** (Up): The direction vector pointing "up" relative to the camera (maps to $+Y$).

Since we want to align the world with these axes, the View Matrix (V) is actually the **inverse** of the camera's world transformation matrix (C).

$$V = C^{-1} = (T \cdot R)^{-1} = R^{-1} \cdot T^{-1}$$

Because the rotation vectors $(\mathbf{r}, \mathbf{u}, \mathbf{f})$ are orthonormal (perpendicular and length 1), the inverse of the rotation matrix is simply its **transpose** (R^T). This leads to a very efficient derivation where we essentially project the world onto the camera's basis vectors using the dot product:

$$V = \underbrace{\begin{pmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -f_x & -f_y & -f_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\text{Rotate (Align Axes)}} \cdot \underbrace{\begin{pmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{pmatrix}}_{\text{Translate (Inverse Position)}}$$

Note: The third row contains $-\mathbf{f}$ (negative forward). This is because in standard OpenGL/WebGL conventions, the camera looks down the **negative** Z-axis. Therefore, the camera's "positive Z" basis vector actually points *backwards*, opposite to the forward vector.

2.4 The Projection Matrix

We now have the scene arranged as seen from the camera's point of view (View Space). However, our screen is 2D, and we need to create the illusion of depth. This is the role of the **Projection Matrix**.

This matrix has two primary responsibilities:

1. **Define the Viewing Volume:** Determine the specific boundaries (l, r, b, t, n, f) of the world that are visible.
2. **Normalisation:** Transform all vertices within that volume into a standardised, cube-shaped volume called **Clip Space**.

In Clip Space, the x, y, and z coordinates for visible geometry must all range from -1 to $+1$. Anything outside this range is "clipped" and discarded. This standardisation is crucial because it allows the GPU to process geometry uniformly, regardless of whether the screen is a tiny smart watch or a massive 4K monitor.

2.4.1 Orthographic Projection

Orthographic projection defines a rectangular box (cuboid) as its viewing volume. It performs a **linear** transformation: it scales the width, height, and depth of the viewing box to fit exactly into the Clip Space cube.

We define the boundaries of this box using six variables:

- **Horizontal Bounds:** l (left plane) and r (right plane).
- **Vertical Bounds:** b (bottom plane) and t (top plane).
- **Depth Bounds:** n (near plane) and f (far plane).

Deriving the Matrix Formulation: To map the arbitrary range $[l, r]$ to the standardised Clip Space range $[-1, 1]$, we perform two logical steps: 1. **Center the Volume:** We shift the

midpoint of the range ($\frac{r+l}{2}$) to the origin 0. 2. **Scale to Size:** We scale the full width of the volume ($r-l$) to the width of clip space (2).

Mathematically, for the X-axis, this results in the equation:

$$x_{clip} = \underbrace{\frac{2}{r-l}}_{\text{Scale Factor}} \cdot x - \underbrace{\frac{r+l}{r-l}}_{\text{Translation}}$$

When we apply this logic to all three axes (x, y, z), we generate the standard Orthographic Matrix:

$$M_{Ortho} = \begin{pmatrix} \frac{\mathbf{2}}{\mathbf{r-l}} & 0 & 0 & -\frac{\mathbf{r+l}}{\mathbf{r-l}} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Notice the bottom row is $\begin{pmatrix} 0 & 0 & 0 & 1 \end{pmatrix}$. This preserves the w component as 1, meaning no perspective distortion occurs; parallel lines remain parallel.

2.4.2 Perspective Projection

Perspective projection simulates the physics of the human eye: objects farther away appear smaller. Its viewing volume is a **frustum** (a truncated pyramid).

We define this frustum using the same symbol names as Orthographic projection, but their geometric meaning is strictly tied to the **Near Plane**:

- n (Near) and f (Far): The positive distances to the near and far clipping planes.
- l, r, b, t: The coordinates of the viewing window **specifically on the Near plane** (at $z = -n$).

Mathematical Formulation: The "Similar Triangle" Rule
Unlike orthographic projection, this is not a linear scaling. To understand why, imagine looking at the scene from the side. We have an object at height y and distance z. We want to project it onto the Near Plane (located at distance n) to find its screen position.

By the geometric property of **Similar Triangles**, the projected height y' is proportional to the ratio of distances:

$$\frac{y'}{n} = \frac{y}{-z} \implies y' = \frac{n \cdot y}{-z}$$

Note the **division by** z. Matrix multiplication can perform addition and multiplication, but it *cannot* perform division directly.

The Matrix and the "Perspective Divide":

We now encounter a fundamental mathematical obstacle. Matrix multiplication is a *linear* operation; it can perform addition (translation) and multiplication (scaling/rotation), but it **cannot** perform division by a variable. However, the geometric rule of Similar Triangles ($y' = y/z$) explicitly requires us to **divide** the x and y coordinates by the depth (z). A standard 4×4 matrix simply has no mechanism to say "divide X by Z".

To solve this, we exploit the full power of **Homogeneous Coordinates**. Up until now, the 4th component (w) of our vectors has strictly remained 1 to allow for translation. Perspective projection changes this rule. It uses the w component as a "storage container" for the depth value.

Notice the bottom row of the Perspective Matrix:

$$\begin{pmatrix} 0 & 0 & \mathbf{-1} & 0 \end{pmatrix}$$

When this matrix multiplies a View-Space vertex $P = (x, y, z, 1)$, the dot product for the last component (w_{clip}) is calculated as:

$$w_{clip} = (0 \cdot x) + (0 \cdot y) + (\mathbf{-1} \cdot z) + (0 \cdot 1) = \mathbf{-z}$$

This effectively "steals" the depth value (z) from the position and stores it in the w component of the resulting vector. (We use $-z$ because in our coordinate system, the camera looks down the negative Z-axis, so $-z$ gives us a positive distance).

The full matrix operation looks like this:

$$M_{Persp} = \begin{pmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & \mathbf{-1} & 0 \end{pmatrix}$$

The resulting vector is in a state known as **Clip Space**. It is not yet the final 2D screen position; it is a 4D vector where the perspective distortion is "pending", waiting in the w component:

$$P_{clip} = M_{Persp} \cdot P_{view} = \left(x_{clip}, y_{clip}, z_{clip}, \mathbf{-z_{view}} \right)$$

The Hardware "Magic": Perspective Division
This is where the GPU hardware takes over. In a fixed-function stage called **Perspective Division** (which happens automatically *after* the Vertex Shader but *before* the Fragment Shader), the GPU iterates over every vertex and divides the x, y, and z components by this w component.

$$P_{NDC} = \left(\frac{x_{clip}}{w_{clip}}, \quad \frac{y_{clip}}{w_{clip}}, \quad \frac{z_{clip}}{w_{clip}} \right) = \left(\frac{x_{clip}}{-z_{view}}, \quad \frac{y_{clip}}{-z_{view}}, \quad \frac{z_{clip}}{-z_{view}} \right)$$

This results in **Normalised Device Coordinates (NDC)**. Because w holds the distance, vertices that are far away (large z) get divided by a large number, shrinking their x and y coordinates towards zero. Vertices close to the camera get divided by a small number, remaining large. This mathematical division is what creates the visual phenomenon of foreshortening that our brains interpret as 3D depth.

2.5 The Final MVP Matrix

We can now combine these three matrices into a single, all-powerful matrix called the **Model-View-Projection (MVP) Matrix**. The multiplication order is critical and must be applied from **right to left**:

$$M_{MVP} = M_{Projection} \cdot M_{View} \cdot M_{Model}$$

1. **Model (M):** Local Space → World Space. (Rotates, scales, and places the object).
2. **View (V):** World Space → View (Camera) Space. (Moves the whole world so the camera is at the origin).
3. **Projection (P):** View Space → Clip Space. (Warps the world to fit the screen and applies perspective).

Why Right-to-Left? In standard mathematical notation, column vectors are placed on the right side of the matrix. Therefore, the vertex P_{local} first interacts with the matrix closest to it (M_{Model}), then the result interacts with M_{View}, and finally $M_{Projection}$.

$$P_{clip} = M_{Projection} \cdot (M_{View} \cdot (M_{Model} \cdot P_{local}))$$

This combined matrix is calculated **once per object** on the CPU and sent to the vertex shader as a single ‘uniform‘. This is highly efficient: instead of performing three separate matrix multiplications for every single vertex (which could be millions), the GPU performs just one.

```
// The 4D result (x, y, z, w) is stored in gl_Position
gl_Position = u_mvpMatrix * a_position;
```

The Post-Shader "Magic": It is important to note that the vertex shader outputs **Clip Space** coordinates (a 4D vector), not screen pixels. Immediately after the vertex shader runs, the GPU automatically performs the **Perspective Divide**: it divides the x, y, and z components by the w component.

$$P_{NDC} = \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)$$

This automatic step is what actually shrinks distant objects (where w is large). Finally, these normalised coordinates are mapped to your screen’s resolution (Viewport Transform) to determine the final pixel location.

2.6 Practical Example: A three.js Scene

High-level libraries like ‘three.js‘ abstract away the manual matrix multiplication for us, but they use these exact same principles. Let’s see how the MVP concepts map to a practical ‘three.js‘ example.

```
import * as THREE from 'three';
// 1. SCENE and RENDERER
const scene = new THREE.Scene();
const renderer = new THREE.WebGLRenderer({ antialias: true });
```

```
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);

// 2. GEOMETRY and MATERIAL
const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshNormalMaterial();
// Colors based on normal vector
const cube = new THREE.Mesh(geometry, material);

// 3. THE MODEL MATRIX (Transformation of the object)
// three.js combines S, R, and T into a single `matrix` property.
// We manipulate it via helper properties: .position, .rotation, .scale.
cube.scale.set(1.5, 1, 1); // Scale (S)
cube.rotation.y = Math.PI / 4;
// Rotate (R)
cube.position.x = 2; // Translate (T)
// three.js will internally compute: Model Matrix = T * R * S
scene.add(cube);

// 4. THE VIEW and PROJECTION MATRICES (The Camera)
// In three.js, the camera object handles both View and Projection.
// PROJECTION setup: new PerspectiveCamera(fov, aspect, near, far)
const fov = 75; // degrees
const aspect = window.innerWidth / window.innerHeight;
const near = 0.1;
const far = 1000;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);

// VIEW setup: Position the camera in the world.
// This defines the inverse transformation that becomes the View Matrix.
camera.position.set(1, 2, 5); // Camera's "eye" position
camera.lookAt(cube.position); // The "target" point we want to look at

// 5. THE RENDER LOOP (Animation)
function animate(time) {
  time *= 0.001;
  // use seconds

  // Animate the cube's rotation (dynamically changing the Model Matrix)
  cube.rotation.x = time;
  cube.rotation.y = time;

  // The magic happens here!
  // renderer.render() tells three.js to:
  // - Update the cube's Model Matrix from its .position, .rotation, .scale
  // - Calculate the View Matrix from the camera's position and target
  // - Get the Projection Matrix from the camera's setup (fov, etc.)
  // - Compute MVP = Projection * View * Model
  // - Send the MVP matrix to the GPU and draw the scene.
  renderer.render(scene, camera);

  requestAnimationFrame(animate);
}

requestAnimationFrame(animate);
```

Listing 3: Full scene setup in three.js

This example shows how a modern graphics library provides a user-friendly API (‘.position‘, ‘.rotation‘, ‘.scale‘, ‘camera.lookAt‘) that directly corresponds to the underlying matrix transformations (Model, View, Projection) that are essential for any real-time 3D graphics.

3 Section 3: The Architecture of Transformations in WebGL

Having established the mathematical foundation of the Model-View-Projection (MVP) matrix, it is crucial to understand how these operations are orchestrated within a real-time rendering engine like `three.js`. The process is a relay race between the Central Processing Unit (CPU) and the Graphics Processing Unit (GPU).

3.1 The Transformation Pipeline: CPU vs. GPU

A common misconception is that all graphics work happens on the graphics card. In reality, the CPU plays a vital role in preparing the data. The pipeline can be broken down into distinct responsibilities:

1. **The Architect (CPU - Scene Graph):** The CPU maintains the logical state of the world. When you move an object in `three.js` (e.g., `cube.position.x += 0.1`), you are updating a node in the **Scene Graph**. The CPU must recalculate the local and world matrices for every object that moved.
2. **The Bridge (Transfer):** When you call `renderer.render()`, the CPU packages these calculated matrices and sends them to the GPU. This data transfer happens via "Uniforms" (global variables like the MVP matrix) and "Attributes" (per-vertex data).
3. **The Factory (GPU - Vertex Shader):** Finally, the GPU receives the matrices and the raw vertex data. It executes the **Vertex Shader** in parallel for every single vertex, performing the actual matrix multiplication ($P_{clip} = M \cdot P_{local}$). This output is in **Clip Space**, which is standardised coordinate system where any geometry falling outside the visible range (-1 to $+1$) is automatically “clipped” (discarded) before the image is drawn.

3.2 Static vs. Dynamic Transformations

In real-time rendering, efficiency is paramount. We strictly distinguish between transformations that are computationally expensive but rare (Static), and those that are cheap but frequent (Dynamic). Understanding this distinction allows us to optimize the "Render Loop" by minimizing the work done every frame.

- **Static (The Setup - The Projection Matrix):** The "Lens" of the camera usually remains constant. Calculating the Projection Matrix involves complex math (frustum planes, aspect ratios, divisions). We compute this **once** during initialization and store it.

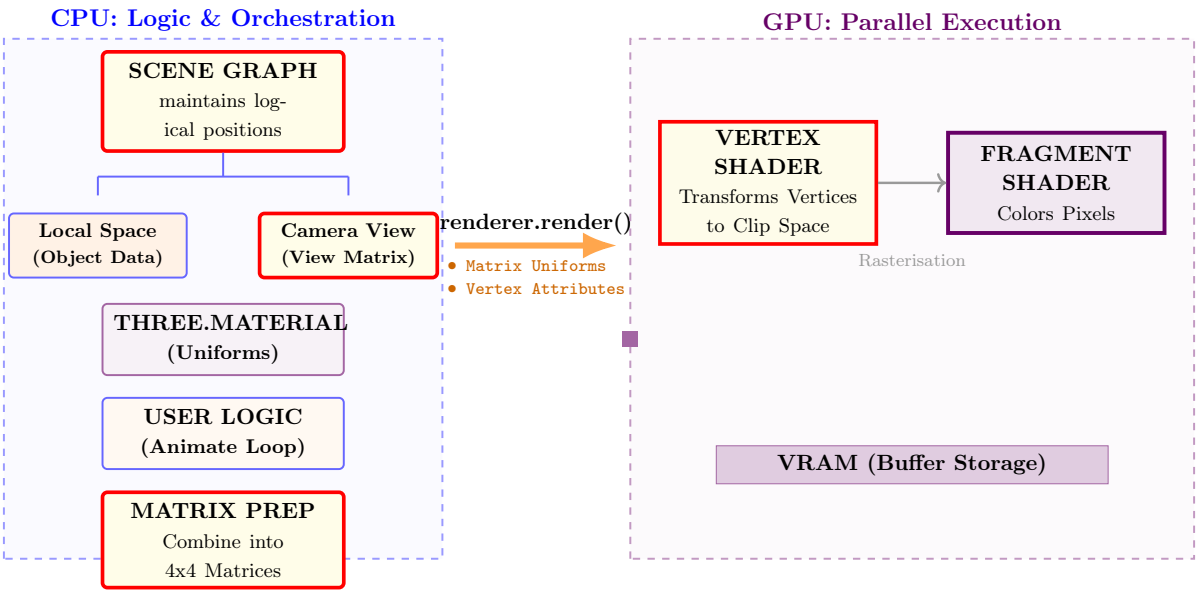

Figure 6: The Architecture of Transformations. The CPU (left) handles the logic and prepares the matrices. The GPU (right) executes the math in parallel.

We only recompute it if the window is resized or the player activates a specific mechanic like a "Zoom".

- **Dynamic (The Loop - Model & View Matrices):** The world is in constant motion.
  - **View Matrix:** Every time the user nudges the mouse, the camera orientation changes. We must recalculate the View Matrix every frame to reflect the new "Inverse World" state.
  - **Model Matrix:** Animated objects (spinning coins, running characters) strictly require their matrices to be updated, multiplied, and uploaded to the GPU 60 times a second.

Phase	Matrix Component	Code Context (three.js)
Setup (Static)	**Projection Matrix** (Lens)	`new THREE.PerspectiveCamera(fov, ...)` *Computed once, cached in memory.*
Loop (Dynamic)	**View Matrix** (Camera Move)	`camera.position.x += speed * delta` *Recomputed every frame from user input.*
Loop (Dynamic)	**Model Matrix** (Object Move)	`mesh.rotation.y += speed * delta` *Recomputed every frame from animation logic.*

Table 1: Frequency of transformation updates. Minimizing calculation in the "Loop" phase is key to maintaining high Frame Rates (FPS).

3.3 Conceptual Mapping: The "Matrix Revelation"

Finally, let us demystify game mechanics. When you play a video game, you are not interacting with physical matter. You are manipulating the variables that feed into these three specific matrices.

The Matrix Revelation

1. The "Mouse Look" is the View Matrix (The Inverse World)
When you move your mouse left to look left, you feel like you are turning your head. Mathematically, the camera stays frozen at $(0, 0, 0)$. Instead, the **View Matrix** rotates the *entire universe* to the right. The trees, terrain, and enemies all orbit around you instantly. Motion is relative; graphics math prefers to move the universe rather than the observer.

2. "Character Animation" is a Hierarchy of Model Matrices
A game character is not a single mesh; it is a tree of nodes (Scene Graph). The "Hand" matrix is multiplied by the "Arm" matrix, which is multiplied by the "Torso" matrix.

$$M_{hand\_world} = M_{torso} \cdot M_{arm} \cdot M_{hand\_local}$$

When you animate the Torso to lean forward, the Hand automatically follows because its final world position is mathematically dependent on the Torso's matrix.

3. The "Sniper Zoom" is the Projection Matrix
Sniper scopes do not move the camera closer to the target (that would be the View Matrix). They simply change the **Field of View (FOV)** variable in the Projection Matrix calculation.

- **Normal Vision:** FOV = 90°. The frustum is wide; objects occupy a small percentage of the screen.
- **Zoomed Scope:** FOV = 10°. The frustum is narrow. The projection math stretches a tiny slice of the world to fill the whole screen, creating the illusion of magnification.

4 Section 4: Reflective Questions: Beyond the Matrix

As we conclude our exploration of transformations, consider the following questions. They are designed to bridge the gap between the mathematics we have covered and the complex challenges you will face in advanced graphics programming.

- **The "Phantom" Hitbox (CPU vs. GPU):** We learned that vertex transformations happen in the Vertex Shader on the GPU. However, game logic (like collision detection) usually runs on the CPU. *If you write a vertex shader that makes a wall pulsate and wave, the CPU doesn't know the wall has moved. How would you handle bullet collisions with a wall that is visually moving on the GPU but physically static on the CPU?*

- **The Problem with Non-Uniform Scaling:** We saw how to scale objects (s_x, s_y, s_z). In the next module, we will learn about lighting, which relies on "normal vectors" (arrows pointing perpendicular to the surface). *If you squash a sphere into a pancake using non-uniform scaling, what happens to those arrows? Do they still point the right way? (Hint: They don't, and this requires a special matrix called the "Inverse Transpose" to fix).*

- **The Euler Angle Trap:** We discussed rotating around the X, Y, and Z axes. This system is called "Euler Angles." *What happens if you rotate 90 degrees on the X-axis, and then try to rotate on the Y-axis? You might find that your Y-axis has now aligned with your Z-axis, causing you to lose a degree of freedom. This is called "Gimbal Lock." How do modern engines avoid this? (Search term: Quaternions).*

- **The Relativity of View:** We established that the camera doesn't move; the world moves around it. *In a multiplayer game, Player A is at* $(10, 0, 0)$ *and Player B is at* $(-10, 0, 0)$. *If the world transformation is unique to the camera, does this mean the GPU must render the scene twice, i.e., once for Player A's view matrix and once for Player B's? How does this impact performance for split-screen gaming or VR?*

"The matrix is everywhere. It is all around us."
— Morpheus, *The Matrix*